MODULE 03 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

# The Messages API

Requests, roles, content blocks, stop reasons, vision and PDFs, and the patterns for getting data in and sources out.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 3 of 36

admin@sthithapragna.com

One request in, one response out. Everything else is what you put in the blocks.

# Anatomy of a request

```
1  {
2    "model": "claude-sonnet-5-5",
3    "max_tokens": 2048,
4    "system": "You help tenants of Copperlet letting agencies.",
5    "messages": [
6      {"role": "user", "content": [
7        {"type": "text", "text": "Is this damp patch my landlord's job?"},
8        {"type": "image", "source": {"type": "file", "file_id": "file_011..."}}
9      ]}
10   ]
11 }
```

4 The system prompt is a top-level field, not a message.

6-8 Content is a list of typed blocks: here text plus an image.

THE TAKEAWAY

Model and max_tokens are required. Everything Claude should see goes in system or messages.

# The request fields that matter

| FIELD | WHAT IT DECIDES |
|---|---|
| `model` | Which model and version answers; required |
| `max_tokens` | Hard cap on output, thinking included; required |
| `system` | Role and standing rules, outside the conversation |
| `messages` | The conversation: alternating user and assistant turns |
| `stream` | Deliver the answer as events instead of all at once |
| `tools, tool_choice` | What Claude may call, and whether it must |
| `stop_sequences` | Strings that end generation early |
| `output_config` | Effort level, and a JSON schema for structured output |

ON THE EXAM

Most exam questions about requests come down to one of these eight fields.

# Roles: user, assistant, and no system message

- **Turns alternate** between user and assistant. Consecutive turns with the same role are merged into one.
- **There is no system role** inside messages. Standing instructions go in the top-level system field.
- **Tool results travel in a user turn,** answering the assistant's tool_use blocks.
- **Prefill is gone:** ending with a partial assistant turn returns a 400 error on Claude 4.6 and later.

> CURRENCY CHECK
>
> To force a format today, use structured outputs, not a prefilled assistant turn.

# Content blocks

| BLOCK | DIRECTION | CARRIES |
|---|---|---|
| `text` | Both | Words |
| `image` | You send | A photo or scan |
| `document` | You send | A PDF or plain text |
| `search_result` | You send | Your retrieved content, with source |
| `tool_result` | You send | The output of a tool you ran |
| `thinking` | Claude sends | Reasoning, with a signature |
| `tool_use` | Claude sends | A request to call your tool |
| `server_tool_use` | Claude sends | A tool Anthropic runs for you |

ON THE EXAM

Read the block type before the text. The type tells your code what to do next.

# Anatomy of a response

```
1  {
2    "id": "msg_01...",
3    "role": "assistant",
4    "model": "claude-sonnet-5-5",
5    "content": [
6      {"type": "thinking", "thinking": "", "signature": "EqQB..."},
7      {"type": "text", "text": "Damp from a leaking gutter is usually the landlord's..."}
8    ],
9    "stop_reason": "end_turn",
10   "usage": {"input_tokens": 2412, "output_tokens": 318,
11             "cache_read_input_tokens": 0}
12 }
```

6 Thinking text is omitted by default on current models; the signature stays.

9 Why generation stopped. Check it before using the answer.

10-11 What you are billed for, cache reads included.

THE TAKEAWAY

Every response also has a request-id header. Log it for tracing and support.

# Stop reasons, and what your code does

| STOP_REASON | MEANING | YOUR CODE |
|---|---|---|
| `end_turn` | Claude finished | Use the answer |
| `max_tokens` | The cap cut it off | Raise the cap or continue |
| `stop_sequence` | One of your strings | Handle as designed |
| `tool_use` | Claude wants a tool | Run it, send tool_result |
| `pause_turn` | Server tool loop paused | Send it back unchanged |
| `refusal` | Claude declined | Fall back; HTTP is still 200 |
| `model_context_window_exceeded` | Window full | Treat as truncated |

ON THE EXAM

HTTP 200 does not mean a complete answer. The stop reason does.

# Vision: images in the request

- **Formats:** JPEG, PNG, GIF (first frame only) and WebP.
- **Volume:** up to 600 images per request on 1M-context models.
- **Size:** 10 MB per base64 image on the Claude API, 5 MB on Bedrock and Google Cloud; max 8000 x 8000 pixels.
- **Cost:** image tokens grow with pixel area, so very large photos cost more without adding useful detail.
- **Sources:** base64, a URL, or a Files API file_id. Bedrock and Google Cloud take base64 only.

> ON THE EXAM
>
> Downscale and compress before sending. It saves tokens and avoids size errors.

# PDFs: the document block

- **Send a PDF as a document block:** base64, a URL, or a file_id.
- **Up to 600 pages** per request on 1M-context models; 32 MB per request.
- **Each page is read as text and as an image,** so tables, charts and stamps count.
- **Cost per page:** typically 1,500 to 3,000 tokens, plus the image cost.
- **Scanned PDFs with no text layer** cannot carry citations.

> ON THE EXAM
>
> A 60-page agreement can cost well over 100,000 input tokens. Send it once, reuse it, cache it.

# Three ways to send a file

| SOURCE | BEST WHEN | WATCH FOR |
|---|---|---|
| `base64 inline` | One-off use; any platform | Resent on every call |
| `URL` | File already public | Not Bedrock or Google Cloud |
| `Files API file_id` | Same file, many requests | Workspace-wide; not Bedrock |

WATCH OUT

Uploaded files are visible to the whole workspace. Never accept a file_id from an end user.

# Citations: answers that point to the source

- **Turn on per document:** citations.enabled - on all documents in a request, or none.
- **Each claim comes back with a pointer:** a page in a PDF, characters in text, or a block in custom content.
- **Guaranteed valid pointers,** and the quoted text is not billed as output.
- **Not with structured outputs:** combining them returns a 400 error.

> THE TIEBREAKER
>
> Need traceable answers? Use citations. Need machine-readable fields? Use a separate structured-output call.

# Search results: citing your own retrieval

- **A search_result block** carries a source, a title and content - your own retrieved passage.
- **Claude cites it** the same way it cites web search results.
- **Send it in a user turn or inside a tool result,** for example from your own search tool.
- **The pattern for RAG:** retrieve, wrap each passage as a search result, get cited answers back.

> ON THE EXAM
>
> Your retrieval system finds the passages. search_result blocks make the answer traceable to them.

# Data access patterns

| YOU NEED | PATTERN |
| --- | --- |
| One document, one question | document block, base64 |
| Same document, many questions | Files API upload, plus prompt caching |
| Answers that point to pages | Citations on the document |
| Cited answers from your own search | search_result blocks |
| Fields a program will import | Structured outputs, in a separate call |
| A Word or Excel file | Convert to text first; not a document type |

ON THE EXAM

The guide calls these Messages API data access patterns. Match the need to the pattern.

# Copperlet's document reader, designed

01
### Upload
Tenancy agreement to the Files API, once.

02
### Ask
Tenant questions reference the file_id, citations on.

03
### Answer
Replies point to the page they came from.

04
### Extract
A separate call with structured outputs for the import fields.

05
### Clean up
Delete or expire the file when the job is done.

THE TAKEAWAY

Two calls, not one: citations for people, structured outputs for programs.

> A developer's tool loop reads each response, shows any text it finds, and ends the conversation.
>
> Sometimes Claude asks to call a tool, and the user is left with a half-finished answer and nothing else happens.

## What should the code branch on after each response?

A. The stop reason: run the tools and send results back on tool use, and finish on end turn

B. Whether the first content block is a text block

C. The HTTP status code, because two hundred means the answer is complete

D. The number of output tokens reported in the usage object

◆ PAUSE AND DECIDE

## What should the code branch on after each response?

**A** The stop reason: run the tools and send results back on tool use, and finish on end turn

The stop reason says exactly why generation ended and what comes next.

**B** Whether the first content block is a text block

*A tool_use response often starts with text. And a thinking block may come first.*

**C** The HTTP status code, because two hundred means the answer is complete

*A tool request, a refusal and a truncation all arrive as 200.*

**D** The number of output tokens reported in the usage object

*Token counts say how much, never why it stopped.*

THE SIGNAL

Branch on stop_reason. It is the API's own statement of what your code must do next.

> Copperlet's letting agents ask about ten questions each afternoon about the same 40-page tenancy agreement. The code base64-encodes the whole PDF into every request.
>
> The team wants to stop resending the same bytes. The feature runs on the first-party Claude API.

## What is the best approach?

A. Paste the agreement's extracted text into the system prompt of every application

B. Upload the agreement once with the Files API and reference its file ID in each request

C. Publish the agreement at a public web address and send that address instead

D. Convert every page into a separate image and send the images instead of the PDF

◆ PAUSE AND DECIDE

## What is the best approach?

A. Paste the agreement's extracted text into the system prompt of every application

*Loses the page images and still resends everything, in every app.*

B. Upload the agreement once with the Files API and reference its file ID in each request

Upload once, reference many times. Exactly what the Files API is for.

C. Publish the agreement at a public web address and send that address instead

*A tenancy agreement is personal data. Never make it public.*

D. Convert every page into a separate image and send the images instead of the PDF

*More work, more bytes, and the text layer is lost.*

THE SIGNAL

"Same document, many requests" on the Claude API is the Files API signal. Cache it too.

> The document reader must do two things with each agreement. Tenants' answers must point to the exact page they came from, and the rent, deposit and start date must be imported into Copperlet's database as fields.
>
> A developer plans one request that does both.

**Which two design choices are right? Choose two.**

- A One request with citations on and a structured output schema, so one call does both jobs
- B Ask Claude to write page numbers in brackets after each claim in its prose answer
- C For tenant answers, a request with citations enabled on the agreement document
- D Lower the temperature so Claude stops inventing page numbers
- E For the import fields, a separate request that uses structured outputs

◆ PAUSE AND DECIDE

## Which two design choices are right? Choose two.

A One request with citations on and a structured output schema, so one call does both jobs

*The API rejects this combination with a 400 error.*

B Ask Claude to write page numbers in brackets after each claim in its prose answer

*Prompted page numbers can be invented. Citations are guaranteed valid.*

C For tenant answers, a request with citations enabled on the agreement document

Built-in citations return valid page pointers.

D Lower the temperature so Claude stops inventing page numbers

*Wrong layer, and current models reject the setting.*

E For the import fields, a separate request that uses structured outputs

Schema-shaped fields for code, in their own call.

THE SIGNAL

Two needs, two features that cannot share a request. Two calls.

> Copperlet's tenant assistant sometimes shows tenants an empty reply bubble. The logs show HTTP 200 responses with a stop reason of refusal.
>
> The code treats every HTTP 200 as a finished answer and displays whatever text it finds.

## What should the developer change?

A Retry the identical request until a response arrives with a stop reason of end turn

B Keep treating any HTTP two hundred as complete, and hide empty bubbles in the interface

C Detect the refusal stop reason, log its details, and show a safe fallback or hand over to a person

D Remove the system prompt so the model has fewer reasons to refuse

◆ PAUSE AND DECIDE

## What should the developer change?

A Retry the identical request until a response arrives with a stop reason of end turn

*Burns calls on a request Claude has already declined.*

B Keep treating any HTTP two hundred as complete, and hide empty bubbles in the interface

*Hides the symptom. The tenant still gets nothing.*

C Detect the refusal stop reason, log its details, and show a safe fallback or hand over to a person

A designed path for a known outcome: evidence logged, tenant helped.

D Remove the system prompt so the model has fewer reasons to refuse

*Throws away the rules that keep the assistant safe and on task.*

THE SIGNAL

A refusal is a normal response type. Give it a designed path, like any other stop reason.

> Letting agents upload inventory photos straight from their phones. Each photo is a high-resolution JPEG of about 14 MB, sent base64-encoded to the Claude API.
>
> Requests with these photos fail, while smaller test images work.

## What should the developer do?

A. Wrap each photo in a PDF so that the larger document limit applies

B. Send each photo in four separate requests, one for each quarter of the image

C. Route photo requests through Amazon Bedrock, which accepts larger images

D. Downscale and compress each photo before sending, keeping it under the per-image size limit

◆ PAUSE AND DECIDE

## What should the developer do?

A Wrap each photo in a PDF so that the larger document limit applies

*A workaround that still sends the same huge image, with no benefit.*

B Send each photo in four separate requests, one for each quarter of the image

*Splits the context Claude needs to judge the room as a whole.*

C Route photo requests through Amazon Bedrock, which accepts larger images

*Backwards: Bedrock's base64 image limit is smaller, 5 MB.*

D Downscale and compress each photo before sending, keeping it under the per-image size limit

Under the 10 MB limit, fewer tokens, and no loss of useful detail.

THE SIGNAL

Smaller test images work, big phone photos fail: a size limit. Fix the input, not the platform.

> A developer moving code from another provider adds a message with the role "system" at the start of the messages list. The API rejects the request.
>
> The developer needs the instructions to apply to the whole conversation.

## Where should the instructions go?

A In a final assistant turn that begins the answer in the required style

B In the top-level system field of the request, outside the messages list

C In a second user message sent before the real question

D Repeated at the start of every user message in the conversation

◆ PAUSE AND DECIDE

## Where should the instructions go?

A In a final assistant turn that begins the answer in the required style

*That is prefill, and current models reject it with a 400.*

B In the top-level system field of the request, outside the messages list

The Messages API's place for standing instructions.

C In a second user message sent before the real question

*Consecutive user turns merge, and it reads as the user talking.*

D Repeated at the start of every user message in the conversation

*Wastes tokens and mixes rules with the user's words.*

THE SIGNAL

There is no system role in messages. Standing rules belong in the system field.

# Module 3 in five lines

- **A request** is model, max_tokens, a system field and alternating turns of typed content blocks.
- **A response** is blocks plus a stop_reason and usage. Branch on the stop reason, not the HTTP code.
- **Images and PDFs** go in as blocks: base64, URL or file_id. Mind size limits and token cost.
- **Same file, many calls:** Files API. Traceable answers: citations or search results.
- **Citations and structured outputs** cannot share a request. Use two calls.

COMING UP

Next: SDKs over REST, and when you need a websocket.